

Laboratory 04

Priority Inversion: See It, Fix It

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXXN236 (ARM Cortex-M33 @ 150 MHz)
RTOS	FreeRTOS v10.6
Toolchain	ARM GNU Toolchain (arm-none-eabi-gcc) + Make
Estimated time	2–3 hours
CLO mapping	CLO 3 & CLO 4

1. Objectives

By completing this lab you will be able to:

1. **Reproduce** the classic three-task priority inversion scenario on real hardware.
2. **Observe** the inversion window in a SEGGER SystemView trace and measure its duration.
3. **Quantify** the difference between theoretical worst-case response time with and without PIP.
4. **Fix** the inversion by replacing a binary semaphore with a FreeRTOS mutex.
5. **Verify** the fix — τ_H 's response time must not exceed τ_L 's critical-section length.

2. Prerequisites

- ☐ Lab 03 complete — SystemView setup working.
- ☐ SEGGER SystemView running and connected to the board.
- ☐ ARM GNU Toolchain and pyocd working.

3. Background

Priority inversion occurs when a high-priority task (τ_H) blocks on a resource held by a low-priority task (τ_L), and a medium-priority task (τ_M) preempts τ_L — causing τ_H to wait for τ_M , even though τ_M shares no resource with τ_H .

Without PIP: Response time of $\tau_H \geq$ duration of τ_M 's execution + τ_L 's remaining critical section. This can be arbitrarily long.

With PIP (FreeRTOS mutex): When τ_H blocks, τ_L inherits τ_H 's priority. τ_M cannot preempt τ_L . τ_H 's blocking is bounded to at most one τ_L critical section.

4. Project Structure

```
lab04_priority_inversion/
  Makefile
  linker_MCXXN236.ld
  src/
    main.c           <- task creation, kernel object init
    inversion_tasks.c/.h <- tauH, tauM, tauL implementations
    FreeRTOSConfig.h
    uart.h / uart.c
    FreeRTOS-Kernel/
```

4.1. Task Parameters

Task	Priority	Period	Critical section
τ_H (high)	3	200 ms	Takes shared resource for 5 ms
τ_M (medium)	2	150 ms	Runs 30 ms of computation (no shared resource)
τ_L (low)	1	300 ms	Holds shared resource for 40 ms

5. Part A — Reproduce the Inversion (Binary Semaphore)

5.1. Step 1 — Implement the Three Tasks

In `inversion_tasks.c`, use a **binary semaphore** (`xSemaphoreCreateBinary`, pre-given to `pdTRUE`) as the shared resource guard:

```

1 void vTaskH(void *pv) /* prio 3 */
2 {
3     TickType_t xLastWake = xTaskGetTickCount();
4     for (;;) {
5         vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(200));
6         TickType_t t_arrive = xTaskGetTickCount();
7         uart_printf("[H] arrived at %lu ms\r\n", t_arrive);
8
9         xSemaphoreTake(xSharedResource, portMAX_DELAY);
10        TickType_t t_acquire = xTaskGetTickCount();
11        uart_printf("[H] acquired at %lu ms (waited %lu ms)\r\n",
12                    t_acquire, t_acquire - t_arrive);
13
14        vTaskDelay(pdMS_TO_TICKS(5)); /* critical section */
15        xSemaphoreGive(xSharedResource);
16        uart_printf("[H] released resource\r\n");
17    }
18 }
19
20 void vTaskM(void *pv) /* prio 2 */
21 {
22     TickType_t xLastWake = xTaskGetTickCount();
23     for (;;) {
24         vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(150));
25         uart_printf("[M] running (no resource held)\r\n");
26         TickType_t start = xTaskGetTickCount();
27         while ((xTaskGetTickCount() - start) < pdMS_TO_TICKS(30)) {}
28         uart_printf("[M] done\r\n");
29     }
30 }
31
32 void vTaskL(void *pv) /* prio 1 */
33 {
34     TickType_t xLastWake = xTaskGetTickCount();
35     for (;;) {
36         vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(300));
37         xSemaphoreTake(xSharedResource, portMAX_DELAY);
38         uart_printf("[L] acquired resource -- holding for 40 ms\r\n");
39         TickType_t start = xTaskGetTickCount();
40         while ((xTaskGetTickCount() - start) < pdMS_TO_TICKS(40)) {}

```

```

41     xSemaphoreGive(xSharedResource);
42     uart_printf("[L] released resource\r\n");
43 }
44 }

```

Note

Tasks are offset at startup so that τ_L acquires the resource first, τ_H arrives while τ_L holds it, and τ_M preempts τ_L shortly after. This is handled in `main.c` using initial `vTaskDelay` offsets.

5.2. Step 2 — Build and Flash

```
make setup && make && make flash
```

5.3. Step 3 — Capture UART Output

Watch for lines where τ_H 's waited X ms value is much larger than the 40 ms τ_L critical section alone.

Expected inversion output:

```

[L] acquired resource -- holding for 40 ms
[H] arrived at 100 ms
[M] running (no resource held)
[M] done
[L] released resource
[H] acquired resource at 170 ms (waited 70 ms)

```

5.4. Step 4 — Capture SystemView Trace

Open SystemView, start recording, let the system run for at least 3 seconds, then stop. In the trace, find an inversion event:

- Identify the moment τ_H blocks on the semaphore.
- Identify the moment τ_M preempts τ_L .
- Measure the **inversion window** — the time τ_H waits while τ_M is running.

Checkpoint A

Annotated SystemView screenshot with the inversion window marked and measured.

6. Part B — Theoretical Analysis

Before fixing the system, compute the worst-case response time analytically.

Without PIP, τ_H 's response time can be bounded as:

$$R_H \leq C_H + B_H + \sum_{j \in \text{hp}(H)} \left\lceil \frac{R_H}{T_j} \right\rceil C_j$$

where B_H is the **blocking term**. Without PIP, B_H is unbounded (τ_M can always preempt τ_L).

With PIP, B_H is bounded to the longest critical section of any lower-priority task sharing the resource:

$$B_H = \max(\text{CS of lower-priority tasks}) = 40 \text{ ms}$$

Question B1

Using the task parameters from the table, apply the RTA recurrence to compute τ_H 's worst-case response time **with PIP**. Show your iteration steps.

? Question B2

From your UART output in Part A, what is the maximum waited X ms value you observed across multiple periods? Does it match the theoretical without-PIP bound?

? Question B3

Compute total utilisation U for the three-task set. Is it schedulable under RMS? Does priority inversion change the schedulability answer? (Hint: yes and no — explain.)

7. Part C — Fix with a Mutex**7.1. Step 1 — Replace Binary Semaphore with Mutex**

In `main.c`, change one line:

```
1 /* BEFORE (binary semaphore -- no PIP): */
2 xSharedResource = xSemaphoreCreateBinary();
3 xSemaphoreGive(xSharedResource);
4
5 /* AFTER (mutex -- PIP enabled): */
6 xSharedResource = xSemaphoreCreateMutex();
```

Also ensure `FreeRTOSConfig.h` has:

```
1 #define configUSE_MUTEXES 1
```

No other code changes are needed — `xSemaphoreTake` and `xSemaphoreGive` have the same API for both mutexes and binary semaphores.

7.2. Step 2 — Rebuild and Flash

```
make && make flash
```

7.3. Step 3 — Observe the Fix

τ_H 's waited time should now be ≤ 40 ms. τ_M cannot preempt τ_L while τ_L holds the mutex.

Expected fixed output:

```
[L] acquired resource -- holding for 40 ms
[H] arrived at 100 ms
[H] acquired resource at 140 ms (waited 40 ms)
[M] running (no resource held)
[M] done
```

7.4. Step 4 — Capture the Fixed SystemView Trace

In the fixed trace, τ_L 's priority **spikes to τ_H 's priority level** when τ_H blocks (visible in SystemView as a priority change event).

✓ Checkpoint C

Annotated SystemView screenshot showing the priority inheritance event (τ_L running at τ_H 's priority).

8. Part D — Experiments**8.1. Experiment 1 — Deep Nesting**

Add a **second** shared resource `xResource2` (also a mutex). Modify τ_L to hold both resources:

```

1 xSemaphoreTake(xSharedResource, portMAX_DELAY);
2 vTaskDelay(pdMS_TO_TICKS(10));
3 xSemaphoreTake(xResource2, portMAX_DELAY);
4 /* ... critical section ... */
5 xSemaphoreGive(xResource2);
6 xSemaphoreGive(xSharedResource);

```

And τ_M also competes for `xResource2`.

? Question D1

FreeRTOS PIP documentation states that the current implementation propagates inheritance only **one level deep** — it does not fully propagate chained inheritance. Describe a scenario with this two-resource design where τ_H still experiences unexpected delay even with mutexes.

8.2. Experiment 2 — ISR Give

Replace the `xSemaphoreGive` inside `vTaskL` with a give from a software timer callback (simulating an ISR-side release).

? Question D2

`xSemaphoreGiveFromISR` does **not** trigger priority inheritance restoration. After τ_L “gives” from the ISR callback, does τ_L ’s priority return to its original level immediately? What does this tell you about using mutexes for resources released from ISR context?

9. Deliverables

#	Item	Details
1	Part A UART capture	At least one inversion event with waited annotated
2	Part A SystemView screenshot	Inversion window marked and measured
3	Part B analysis	RTA calculation with and without PIP
4	Part C UART capture	waited bounded to ≤ 40 ms
5	Part C SystemView screenshot	Priority inheritance event visible
6	Written answers	Questions B1–B3, D1–D2

10. Key Concepts Demonstrated

Concept	Where you saw it
Three-task priority inversion scenario	Part A
Inversion window measurement	Part A SystemView trace
RTA blocking term B_H	Part B analysis
Mutex vs. binary semaphore for PIP	Part C code change
Priority inheritance in SystemView	Part C trace
Limits of FreeRTOS PIP (one-level)	Experiment D1

11. Reference

Resource	Relevant sections
Sha, Rajkumar & Lehoczky (1990)	“Priority Inheritance Protocols” (<i>IEEE Trans. Computers</i>)
Buttazzo, <i>Hard Real-Time Computing Systems</i>	Ch. 7.1–7.3
FreeRTOS docs	<code>xSemaphoreCreateMutex</code> , <code>configUSE_MUTEXES</code>
SEGGER SystemView manual	Priority display, event annotations
